Introducción a la Programación
Algoritmos y Estructuras de Datos I

Segundo cuatrimestre de 2026

Departamento de Computación - FCEyN - UBA

Programación Imperativa: Depuración de código - Estructuras de Datos
(Arreglos, Listas, Pilas, Colas, Diccionarios, Archivos)

IP - AED I: Temario de la clase

- Repaso del Transformación de Estados y Ejecución Simbólica
- ¿Qué es depurar el código (*debbugear*)? ¿Para qué sirve?
- Debugging en VSCode
- Estructuras de Datos y Tipos Abstractos de Datos (TAD)
- Arreglos
- Arreglos vs Listas
- Arreglos en Python
- Operaciones báscias de arreglos
- Listas en Pyhton
- Operaciones báscias de listas. ¿Cómo recorrer listas?
- Modelado de matrices. ¿Cómo recorrer una matriz?
- Iterables y uso de `For`
- TAD Pila en Python utilizando el tipo de dato List
- TAD Cola en Pyhton utilizando el tipo de dato List
- TAD Pila utilizando LifoQueue
- TAD Cola utilizando Queue
- TAD Diccionario
- Manejo de archivos

Variables en imperativo

Repasando

- Nombre asociado a un espacio de memoria.
- La variable puede tomar distintos valores a lor largo de la ejecución.
- En Python se declaran dando su nombre (y opcionalmente su tipo)
  `x: int` # *x* es una variable de tipo int.
  `c: str` # *c* es una variable de tipo string.
- Programación imperativa:
  - Conjunto de variables.
  - Instrucciones que van cambiando sus valores.
  - Los valores finales, deberían resolver el problema.

Transformación de estados

Repasando

- Llamamos estado de un programa a los valores de todas sus variables en un punto de su ejecución:
  - Antes de ejecutar la primera instrucción.
  - Entre dos instrucciones.
  - Después de ejecutar la última instrucción.
- Veremos la ejecución de un programa como una sucesión de estados.
- La asignación es la instrucción que transforma estados.
- El resto de las instrucciones son de control: modifican el flujo de ejecución es decir, el orden de ejecución de las instrucciones.

Ejemplo de *transformación de estados*

Repasando

```
def ejemplo() -> int:
  x: int = 0
  x = x + 3
  x = 2 * x
  return x
```

Ejemplo de transformación de estados:

```
x = 0
  //Estado 1 x == 0
x = x + 3
  //Estado 2 x == 3
x = 2 * x
  //Estado 3 x == 6
```


Ejecución simbólica

Repasando

```
def suc(x: int) -> int:
        //estado a;
    x = x + 2
        //estado b
        //vale x == x@a+2;
        «En el estado b, x vale lo que valía en el estado a más 2»
    x = x - 1
        //estado c
        //vale x == x@b-1;
        «En el estado c, x vale lo que valía en el estado b menos 1»
    return x
```

- De esta manera, mediante la transformación de estados, podremos realizar una ejecución simbólica del programa, declarando cuánto vale cada variable, en cada estado del programa, en función de los valores anteriores.
- Algunas técnicas de verificación estática utilizan estos recursos.

Debugging

- La mayoría de los IDEs nos brindan una herramienta MUY poderosa llamada **DEBUG**
- Con esta herramienta vamos a poder ir siguiendo la ejecución del programa *paso a paso*
- Esta herramienta nos permite:
  - ir siguiendo el flujo de ejecución de las instrucciones
  - ir visualizando como las instrucciones (asignaciones) del programa van transformando los estados
- La ejecución simbólica se vuelve real y podemos ver la evolución concreta de cada variable
- Es una herramienta fundamental para encontrar errores (BUGS) en nuestro código, es decir, para depurar el código.

¿Qué es Debugging y para qué sirve?

1. Podemos ir paso a paso analizando los valores de las variables durante la ejecución (antes y después de cada instrucción)
2. Sirve para poder realizar seguimiento del código y encontrar errores
3. Podemos avanzar paso a paso o saltar al siguiente breakpoint
4. Podemos terminar la ejecución por la mitad o bien continuar hasta el final
5. Con VSCode podemos agregar breakpoints durante el momento de debugging, o eliminarlos
6. Se pueden agregar breakpoints con condiciones lógicas, por ejemplo: valor_actual = 7

Agregar un breakpoint (punto de detención) en el código

Debemos hacer click a la izquierda del número de línea para agregar el punto de detención en esa línea:

```
1
2  def suma_total(s:[int])-> int:
3      total:int = 0
4      indice_actual:int = 0
5      longitud:int = len(s)
6
7      while (indice_actual < longitud):
8          valor_actual:int = s[indice_actual]
9          total = total + valor_actual
10         indice_actual += 1
11
12     return total
13
```

Figura: Agregamos un breakpoint en la línea 7 del código

Ejecutar con Debug

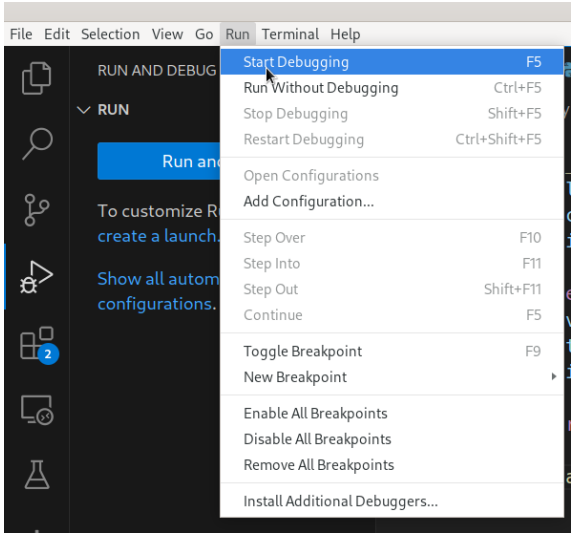


Figura: Ejecutamos el código con la opción Debug

Usamos los controles de la IDE para desplazarnos

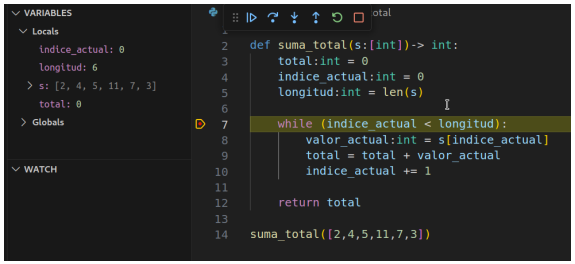


Figura: Podemos ver las variables con sus valores al momento del break y usar los controles para movernos

Usamos los controles de la IDE para desplazarnos



- F5 Continuar hasta el siguiente breakpoint (o si no hay más hasta el final)
- F10 Siguiente paso salteando ingresar a la función que se esté evaluando en esta línea
- F11 Siguiente paso ingresando a la función que se esté evaluando en esa línea
- Shift+F11 Salir de la evaluación de la función a la que se ingresó
- Ctrl + Shift + F5 Reiniciar el debug desde el principio
- Shift + F5 Detener el debugging

Estructuras de Datos: Arreglos

- Secuencias de una cantidad fija de valores del mismo tipo.
- Se declaran con un nombre y un tipo:
  - Según el lenguaje, además se debe indicar su tamaño (el cual permanece fijo).
  - Veremos que en Python, los arrays tienen longitud variable.
- Solamente hay valores en las posiciones válidas (dentro de su tamaño).
- Una sóla variable contiene muchos valores:
  - A cada valor se lo accede directamente mediante corchetes.
  - Si *a* es un arreglo de 10 elementos, *a*[5] devuelve el 6to valor.
  - El primer elemento es el de índice 0.

Arreglos

- Supongamos que la variable *a* tiene tipo de dato arreglo de int.
- Podemos ejemplificar que sucede con su referencia en esta simplificación:
  - La variable *a* tiene su referencia en B1.
  - Como es un arreglo de tamaño 4, tiene asociadas 4 posiciones más de memoria.
  - Por ejemplo: *a*[2] tiene el valor de 7 (el valor que está en B3).
  - Si tenemos que describir el estado de la variable *a*, el mismo es [5,6,7,8].
    - *a*[2] no es una variable en sí misma, la variable es *a*.

Memoria

	1	2	3	4	5
A					
B	5	6	7	8	
C					
D					
E					

Variables

Nombre	Tipo	Tamaño	Valor	Referencia
a	Array de Int	4	[5,6,7,8]	B1

Arreglos y Listas

- Ambos representan secuencias de elementos de un tipo.
- Los arreglos suelen tener longitud fija, las listas, no.
- Los elementos de un arreglo pueden accederse en forma independiente y directa:
  - Los de la lista se acceden secuencialmente, empezando por la cabeza,
  - Para acceder al i-ésimo elemento de una lista, hay que obtener i veces la cola y luego la cabeza.
  - Para acceder al i-ésimo elemento de un arreglo, simplemente se usa el índice.

Memoria

	1	2	3	4	5
A			7		
B	5				
C					8
D		6			
E					

Variables

Nombre	Tipo	Tamaño	Valor	Referencia
a	Lista de int		[5,6,7,8]	B1

Arreglos Python

- Se requiere un importar un módulo *array* que permiten utilizar arreglos (otra posibilidad es NumPy).
- Los arreglos en Python están diseñados para trabajar con datos numéricos (enteros, flotantes, etc.)
- Al crear el arreglo, se indica su tipo y opcionalmente su contenido inicial.
- Solo pueden contener elementos de un mismo tipo.

```
from array import *
a: array = array(typecode, [inicializers])
```

```
from array import *
a: array = array('i', [10, 20, 30])
```

Arreglos Python

- Tipos de datos del módulo *array*

Code Type	Python Type	Full Form	Tamaño (en Bytes)
u	unicode character	Python Unicode	2
b	int	Signed Char	1
B	int	Unsigned Char	1
h	int	Signed Short	2
l	int	Signed Long	4
L	int	Unsigned Long	4
q	int	Signed Long Long	8
Q	int	Unsigned Long Long	8
H	int	Unsigned Short	2
f	float	Float	4
d	float	Double	8
i	int	Signed Int	2
I	int	Unsigned Int	2

Arreglos Python

Operaciones básicas sobre arrays

Sea a un array:

$a[i] \longrightarrow$ obtiene el valor del elemento i de a

$a[i] = x \longrightarrow$ asigna x en el elemento i de a

a.append(x) $\longrightarrow$ añade x como nuevo elemento de a

a.remove(x) $\longrightarrow$ elimina el primer elemento en a que coincida con x

a.index(x) $\longrightarrow$ obtiene la posición donde aparece por primera vez el elemento x

a.count(x) $\longrightarrow$ devuelve la cantidad de apariciones del elemento x

a.insert(p,x) $\longrightarrow$ inserta el elemento x delante de la posición p

Listas en Python

- En Python, las listas pueden tener elementos de diferentes tipos de datos.
- Son parte de las estructuras de datos integradas de Python, no necesitan importar ningún módulo adicional.
- Al igual que los arrays en Python, tienen tamaño dinámico.
- Son un poco menos eficientes que los arreglos en términos de memoria y rendimiento cuando se trabaja con grandes volúmenes de datos numéricos.

¿Cómo se declaran?

- `variableLista = [] #lista vacia`
- `otraVariableLista = list() #lista vacia`
- `otraVariable = [1, 2, True, 'Hola', 5.8]`
- `unaMas = list([4, 9, False, 'texto'])`

Listas en Python

Operaciones básicas sobre listas

Las listas y los arrays comparten muchas operaciones.

Sea a una lista:

$a[i] \longrightarrow$ obtiene el valor del elemento i de a

$a[i] = x \longrightarrow$ asigna x en el elemento i de a

a.append(x) $\longrightarrow$ añade x como nuevo elemento de a

a.remove(x) $\longrightarrow$ elimina el primer elemento en a que coincida con x

a.index(x) $\longrightarrow$ obtiene la posición donde aparece por primera vez el elemento x

a.count(x) $\longrightarrow$ devuelve la cantidad de apariciones del elemento x

a.insert(p, x) $\longrightarrow$ inserta el elemento x delante de la posición p

Listas en Python

Operaciones básicas sobre listas

Un ejemplo de una operación que tiene el array y no la lista:
Buffer Info devuelve una tupla con dirección de memoria actual de los arrays y número de elementos (Útil sólo para operaciones de bajo nivel).

operaciones_arrays_listas.py > ...

```python
# Importar módulo array
import array

# Declarar lista con valores numéricos
lista1 = [1, 0, 1, 0, 1, 1, 0, 0]

# Declarar 'array1' de tipo 'char sin signo' con datos de 'lista1'
array1 = array.array('B', lista1)

print("Buffer info: " + str(array1.buffer_info()))
print("Buffer info: " + str(lista1.buffer_info()))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
Buffer info: (2071285481920, 8)
Traceback (most recent call last):
  File "c:\@Martin\Facultad\intro-programacion\teoricas\t12 - listas\ejemplos\operaciones_arrays
_listas.py", line 11, in <module>
    print("Buffer info: " + str(lista1.buffer_info()))
                                ^^^^^^^^^^^^^^^^^^
AttributeError: 'list' object has no attribute 'buffer_info'
```


Listas en Python

Como recorrer una lista

Podemos utilizar las distintas estructuras de control de ciclo para recorrer los elementos de una lista utilizando índices:

```python
edades: list = [20, 41, 6, 18, 23]

# Recorriendo los índices
for i in range(len(edades)):
    print(edades[i])

# Con while y los índices
indice = 0
while indice < len(edades):
    print(edades[indice])
    indice += 1
```


Listas en Python

Como recorrer una lista

También podremos utilizar el for para recorrer directamente sus elementos:

```python
edades: list = [20, 41, 6, 18, 23]

# Recorriendo los elementos
for edad in edades:
    print(edad)
```


Listas en Python

Matrices

Podemos pensar una matriz como una lista de listas. Podemos recorrerlas a través de sus índices:

```python
ganadores = [['Messi', 'Cristiano', 'Mbappe'], [7, 5, 1]]

# Recorriendo los índices
# i serían las filas
print("++ Con for - i son filas ++")
for i in range(len(ganadores)):
    for j in range(len(ganadores[i])):
        print("ganadores["+str(i)+"]["+str(j)+"] = " + str(ganadores[i][j]))

print("++ Con while y los índices ++")
# Con while y los índices
fila = 0

while fila < len(ganadores):
    columna = 0
    while columna < len(ganadores[fila]):
        print("ganadores["+str(fila)+"]["+str(columna)+"] = " + str(ganadores[fila][columna]))
        columna += 1
    fila += 1
```

Listas en Python

Iterables

En Python, aquellas variables cuyo tipo de dato sea *iterable* pueden ser recorridas con un for.

NOTA (para quienes saben Python): los conceptos de iterable e iterators están fuera del temario de la materia.

```python
edades: list = [20, 41, 6, 18, 23]

# Recorriendo los elementos
for edad in edades:
    print(edad)
```


Tipos Abstractos de Datos

Un Tipo Abstracto de Datos (TAD) es un modelo que define valores y las operaciones que se pueden realizan sobre ellos.

- Se denomina abstracto ya que la intención es que quien lo utiliza, no necesita conocer los detalles de la representación interna o bien el cómo están implementadas sus operaciones.

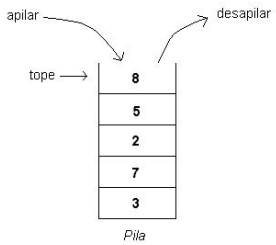
El tipo lista que estuvimos viendo es un TAD:

- Se define como una serie de elementos consecutivos
- Tiene diferentes operaciones asociadas: append, remove, etc
- Desconocemos cómo se usa/guarda la información almacenada dentro del tipo

TAD Pila en Python usando List

Una pila es una lista de elementos de la cual se puede extraer el último elemento insertado.

- También se conocen como listas LIFO (Last In - First Out / el último que entra es el primero que sale)
- Operaciones básicas
  - apilar: ingresa un elemento a la pila
  - desapilar: saca el último elemento insertado
  - tope: devuelve (sin sacar) el ultimo elemento insertado
  - vacia: retorna verdadero si está vacía



TAD Pila en Python usando List

Ejemplos de problemas que naturalmente se modelarían con una pila

- Una pila de platos acumulados en la bacha esperando a ser lavados (no se puede sacar los de abajo, sin antes lavar los ultimos apoyados).
- Una pila de libros o meter y sacar libros de una caja.
- En las góndolas del supermercado, el fondo del estando es el fondo de la pila, y el tope son los artículos que se pueden tomar facilmente.
- Ponerse muchas remeras, una arriba de la otra.

TAD Pila en Python usando List

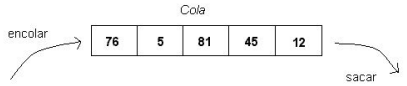
En Python, el tipo lista provee los métodos necesarios para poder usar una lista como una pila

- Operaciones básicas
  - apilar: ingresa un elemento a la pila
    - append
  - desapilar: saca el último elemento insertado
    - pop
  - tope: devuelve (sin sacar) el ultimo elemento insertado
    - a[-1]
  - vacia: retorna verdadero si está vacía
    - len(a)==0

TAD Cola en Python usando List

Una cola es una lista de elementos en donde siempre se insertan nuevos elementos al final de la lista y se extraen elementos desde el inicio de la lista.

- También se conocen como listas FIFO (First In - First Out / el primero que entra es el primero que sale)
- Operaciones básicas
  - encolar: ingresa un elemento a la cola
  - sacar: saca el primer elemento insertado
  - vacia: retorna verdadero si está vacía



TAD Cola en Python usando List

Ejemplos de problemas que naturalmente se modelarían con una cola

- La fila en la parada de colectivos
- La fila de la caja de un supermercado
- La fija en la cabina de peaje

TAD Cola en Python usando List

En Python, el tipo lista provee los métodos necesarios para poder usar una lista como una cola

- Operaciones básicas
  - encolar: ingresa un elemento a la pila
    - append
  - desencolar: saca el primer elemento insertado
    - pop(0)
  - vacia: retorna verdadero si está vacía
    - len(a)==0

TAD Pila en Python usando LifoQueue

- En Python, el tipo lista provee los métodos necesarios para poder usar una lista como una pila
- También, podemos importar el tipo LifoQueue del módulo queue, que nos da una implementación de Pila

from queue import LifoQueue
pila = LifoQueue()

- Operaciones implementadas en el tipo:
  - apilar: ingresa un elemento a la cola
    - put
  - desapilar: devuelve y quita el último elemento insertado
    - get
  - tope: devuelve (sin sacar) el ultimo elemento insertado
    - No está implementado
  - vacia: retorna verdadero si está vacía
    - empty

TAD Cola en Pyhton usando Queue

- En Python, el tipo lista provee los métodos necesarios para poder usar una lista como una cola
- También, podemos importar el tipo Queue del módulo queue, que nos da una implementación de Cola

from queue import Queue
cola = Queue()

- Operaciones implementadas en el tipo:
  - encolar: ingresa un elemento a la cola
    - put
  - desencolar: saca el primer elemento insertado
    - get
  - vacia: retorna verdadero si está vacía
    - empty

TAD Diccionario en Python

Un diccionario es una estructura de datos que permite almacenar y organizar pares clave-valor.

- Las claves deben ser inmutables (como cadenas de texto, números, etc), mientras que los valores pueden ser de cualquier tipo de dato.
- La clave actúa como un identificador único para acceder a su valor correspondiente.
- Los diccionarios son mutables, lo que significa que se pueden modificar agregando, eliminando o actualizando elementos.
- No ordenados: Los elementos dentro de un diccionario no tienen un orden específico. No se garantiza que se mantenga el orden de inserción de los elementos.

diccionario = clave1:valor2, clave2:valor2, clave3:valor3

- Operaciones basicas de un diccionario:
  - Agregar un nuevo par Clave-Valor
  - Eliminar un elemento
  - Modificar el valor de un elemento
  - Verificar si existe una clave guardada
  - Obtener todas las claves
  - Obtener todas los elementos

TAD Diccionario en Python

Un diccionario es una estructura de datos que permite almacenar y organizar pares clave-valor.

- El valor puede ser cualquier tipo de dato, en particular podría ser otro diccionario

```
infoPaisFrancia = {'Capital':'París',
                   'Campeonatos de Mundo':2}

infoPaisArgentina = {'Capital':'Buenos Aires',
                     'Campeonatos de Mundo':3}

infoPaisChile = {'Capital':'Santiago',
                 'Campeonatos de Mundo':0}

infoPaises = {'Chile': infoPaisChile ,
              'Argentina': infoPaisArgentina,
              'Francia':infoPaisFrancia}
```

Manejo de Archivos en Python

El manejo de archivos, también puede pensarse mediante la abstracción que nos brindan los TADs

- Necesitamos una operación que nos permita abrir un archivo
- Necesitamos una operación que nos permita leer sus lineas
- Necesitamos una operación que nos permita cerrar un archivo

```
# Abrir un archivo en modo lectura
archivo = open("archivo.txt", "r")

# Leer el contenido del archivo
contenido = archivo.read()
print(contenido)

# Cerrar el archivo
archivo.close()
```


Manejo de Archivos en Python

archivo = open("PATH AL ARCHIVO", MODO, ENCODING)

- Algunos de los modos posibles son: escritura (w), lectura (r), texto (t - es el default)
- El encoding se refiere a como está codificado el archivo: UTF-8 o ASCII son los más frecuentes.

Operaciones básicas

- Lectura de contenido:
  - read(size): Lee y devuelve una cantidad específica de caracteres o bytes del archivo. Si no se especifica el tamaño, se lee el contenido completo.
  - readline(): Lee y devuelve la siguiente línea del archivo.
  - readlines(): Lee todas las líneas del archivo y las devuelve como una lista.
- Escritura de contenido:
  - write(texto): Escribe un texto en el archivo en la posición actual del puntero. Si el archivo ya contiene contenido, se sobrescribe.
  - writelines(lineas): Escribe una lista de líneas en el archivo. Cada línea debe terminar con un salto de línea explícito.

¿Podremos implementar este problema?

```
problema invertirTexto(in archivoOrigen: string, in archivoDestino: string) : {
  requiere: {El archivo archivoOrigen debe existir.}
  asegura: {Se crea un archivo llamado archivoDestino cuyo contenido será el resultado de hacer un reverse en cada una de sus filas}
  asegura: {Si el archivo archivoDestino existia, se borrará todo su contenido anterior}
}
```